3

CPU moss
StarryOS Futex

3.1

3.1.1 StarryOS

StarryOS `axtask` `axsched`
RR CPU
CPU
StarryOS Future Waker I/O
Futex WAIT
WAKE robust list

3.1.2 base os

Futex Waker
Waker

`FUTEX_WAIT_BITSET`

`waitid` pidfd

3.1.3 moss

moss StarryOS RR

	StarryOS	moss
	RR CPU	
	Future Waker	PollSet
		TimerKey
Futex	WAIT WAKE	
	`waitpid`	`waitid` pidfd

3-1

moss
Futex

3.2

3.2.1

- `Running` CPU
- `Ready`
- `Blocked`
- `Exited`

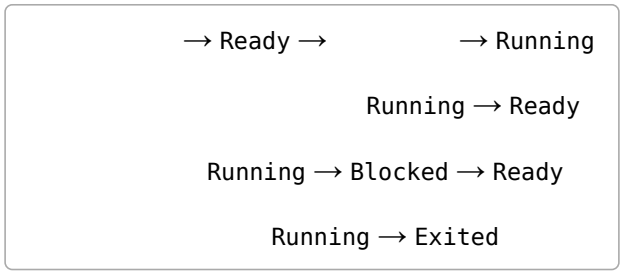
```rust
pub enum TaskState {
    Running = 1, // 正在 CPU 上执行
    Ready   = 2, // 可以运行，正在队列中等待
    Blocked = 3, // 正在等待事件，暂时不能运行
    Exited  = 4, // 已经结束，等待资源回收
}

pub struct TaskInner {
    // 调度器通过该字段判断任务当前能否运行
    state: AtomicU8,
    // ...
}
```

3-1

`state` Futex

`Blocked` `Ready`



3-2

`Ready` `Running` CPU

`Ready` `Blocked`

moss Futex

`Blocked` `Ready` moss

3.2.2

RR CPU CPU

`Ready` 5

CPU

scheduler_timer_tick



CPU

CPU idle

3.2.3

resched CPU

idle CPU

```
fn resched(&mut self) {
    // 从就绪队列选择任务；队列为空时运行 idle 任务
    let next = self.scheduler.lock().pick_next_task()
        .unwrap_or_else(|| unsafe {
            IDLE_TASK.current_ref_raw().get_unchecked().clone()
        });

    self.switch_to(crate::current(), next);
}

fn switch_to(&mut self, prev: CurrentTask, next: AxTaskRef) {
    // 将选中的任务设为正在运行
    next.set_state(TaskState::Running);

    // 保存当前任务现场，恢复下一个任务现场
    let prev_ctx = prev.ctx_mut_ptr();
    let next_ctx = next.ctx_mut_ptr();
    unsafe { (*prev_ctx).switch_to(&*next_ctx) };
}
```

3-3

switch_to Running “ ”

TaskContext

CPU TaskContext

moss TaskContext

UserContext

3.2.2

→ → → →

3.2.4 CPU

CPU cpumask CPU
CPU 0 CPU 2 CPU
CPU cpumask
CPU CPU

```
fn select_run_queue_index(cpumask: AxCpuMask) -> usize {
    loop {
        // 在所有 CPU 编号之间轮转
        let index = RUN_QUEUE_INDEX.fetch_add(1, Ordering::SeqCst)
            % axconfig::plat::CPU_NUM;
        // 只选择亲和性掩码允许的 CPU
        if cpumask.get(index) {
            return index;
        }
    }
}
```

3-4 CPU
CPU CPU
CPU
CPU moss
CPU 1
Linux sched_getaffinity sched_setaffinity

```
// 用户位图中的每一位对应一个 CPU
for cpu_id in 0..axconfig::plat::CPU_NUM {
    if user_mask[cpu_id / 8] & (1 << (cpu_id % 8)) != 0 {
        cpu_mask.set(cpu_id, true);
    }
}

// 保存新的亲和性范围
let curr = current().clone();
curr.set_cpumask(cpu_mask);

// 当前 CPU 已不在范围内时，将任务迁移到允许的 CPU
if !cpu_mask.get(axhal::percpu::this_cpu_id()) {
    let migration_task = TaskInner::new(
        move || migrate_entry(curr),
        "migration-task".into(),
        4096,
    ).into_arc();
    current_run_queue().migrate_current(migration_task);
}
```

3-5 CPU

CPU CPU

moss

`sched_yield` CPU moss

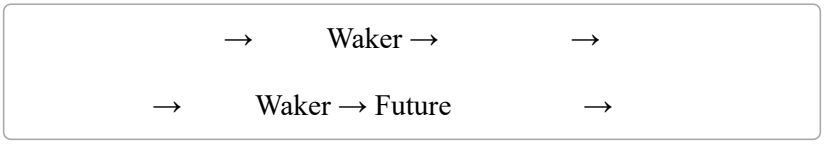
3.3

3.3.1

moss Future “ ” Waker “ ”

Waker

Future “ ” Waker



3-1

Waker “ Waker ”

Waker “ ”

3.3.2 Waker PollSet

Waker StarryOS

Waker `wake()` moss

timerfd pidfd Futex

`PollSet` Waker eventfd timerfd pidfd

moss `has_waiters`

Waker

```rust
pub struct PollSet {
    inner: Lazy<Mutex<Inner>>,
    // 没有等待者时，wake 可以直接返回
    has_waiters: AtomicBool,
}

pub fn wake(&self) -> usize {
    if !self.has_waiters.load(Ordering::Acquire) {
        return 0;
    }
    // 取出并清空本轮登记的 Waker
    let (wakers, len) = self.inner.lock().take_wakers();
    self.has_waiters.store(false, Ordering::Release);
    // 真正唤醒时已经不再持有内部锁
    for waker in wakers.iter().take(len) {
        unsafe { waker.assume_init_read() }.wake();
    }
    len
}
```

3-6 PollSet

`take_wakers` Waker `wake()`

PollSet

PollSet “ ”

Futex

3.3.3 block_on

Future “ ”

`block_on` Future `Ready` `Pending`

CPU 3.2.3

```rust
fn wake_by_ref(self: &Arc<Self>) {
    if let Some(task) = self.task.upgrade() {
        // 先记录通知已经到达
        *self.woke.lock() = true;
        // 再把阻塞任务放回合适的运行队列
        select_run_queue(&task).unblock_task(task, false);
    }
}

match future.poll(&mut context) {
    // 没收到通知，当前任务可以安全进入 Blocked
    Poll::Pending if !*woke => run_queue.blocked_resched(woke),
    // 通知先到，不再阻塞，只让其他任务先运行一次
    Poll::Pending => yield_now(),
    Poll::Ready(output) => return output,
}
```

3-7 Waker block_on

Pending blocked_resched unblock_task

woke

woke CPU Future

Future

interruptible Future

EINTR

Waker PollSet Waker Futex

3.5

3.4

3.3 “ ”

“ ”

3.4.1

“ ”

moss

FUTEX_WAIT_BITSET

nanosleep clock_nanosleep

TIMER_ABSTIME moss

EINTR

3.4.2

StarryOS

moss `BTreeMap` `deadline` `seq` `TimerKey` alarm task

```
struct TimerKey {
    // 定时事件应当触发的时刻
    deadline: Duration,
    // 区分截止时间相同的多个事件
    seq: u64,
}

struct AlarmRuntime {
    // 为新事件生成不重复的序号
    next_seq: u64,
    // 按到期顺序保存全部定时事件
    timers: BTreeMap<TimerKey, Entry>,
}
```

3-8

moss `seq` Key Key Key alarm task

`TimeManager`

`ITIMER_REAL` `SIGALRM` `ITIMER_VIRTUAL` `ITIMER_PROF`

3.4.3

`ProcessData`

wait

Waker `child_exit_event`

```rust
// 先检查是否已有子进程状态可以报告
match check_children().transpose() {
    Some(result) => Poll::Ready(result),
    None => {
        // 暂无结果，登记父进程的 Waker
        child_exit_event.register(cx.waker());
        // 登记后复查，覆盖同时发生的状态变化
        match check_children().transpose() {
            Some(result) => Poll::Ready(result),
            None => Poll::Pending,
        }
    }
}
```

3-9

Shell

wait

3.4.4 wait pidfd

moss

Pid Pgid Any

ECHILD

WNOHANG	
WUNTRACED	
WCONTINUED	
WNOWAIT	

3-2

waitpid wait4 waitid siginfo_t

moss waitid PID

pidfd

pidfd moss pidfd waitid

PID WNOWAIT

3.5 Futex

3.5.1

Futex

moss FutexKey Futex

Futex

Futex owner_dead

moss

```
pub struct WaitQueue {
    // 每个等待项依次保存：
    // 通知状态、唤醒位掩码和任务唤醒入口
    queue: SpinNoIrq<
        VecDeque<(Arc<AtomicBool>, u32, Arc<PollSet>)>
    >,
}
```

3-10 Futex

AtomicBool notified u32 bitset PollSet

“ wake ”

3.5.2 WAIT WAKE

moss FUTEX_WAIT

FUTEX_WAKE EAGAIN

Future FUTEX_WAKE

notified PollSet

wake

```
// 第一次检查：通知可能已经先于登记到达
if notified.load(Ordering::Acquire) {
    return Poll::Ready(());
}

// 条件未满足，登记当前任务的 Waker
poll.register(cx.waker());

// 第二次检查：覆盖“检查与登记之间”的竞争窗口
if notified.load(Ordering::Acquire) {
    Poll::Ready(())
} else {
    Poll::Pending
}
```

3-11 Futex Waker

notified Waker

Waker notified

moss Waker

3.5.3

`notified` `FUTEX_WAIT_BITSET` `FUTEX_WAKE_BITSET` bitset mask

`FUTEX_REQUEUE` Futex `FUTEX_CMP_REQUEUE` `EAGAIN` `notified` bitset PollSet Waker pthread

3.5.4

Futex moss `interruptible(timeout(...))` `ETIMEDOUT` `EINTR` `notified` `FUTEX_WAIT` `FUTEX_WAIT_BITSET` moss Linux `FUTEX_CLOCK_REALTIME`

LoongArch64 `futex_wait` pthread cancellation

3.5.5 robust list

moss Futex `set_robust_list` robust list Futex moss `futex_offset` Futex `owner_dead` `EOWNERDEAD` `ROBUST_LIST_LIMIT` `clear_child_tid` Futex `pthread_join`

3.6

StarryOS RR moss PollSet `TimerKey` moss Futex robust list `waitid` pidfd